



SIMULACIONES Y PROCESOS ESTOCÁSTICOS

2026

Teoría: Dr. Alejandro Mezio – mezioalejandro@uca.edu.ar

Práctica: Dr. Luis Herrera – lherrera@uca.edu.ar

REMUESTREO: BOOTSTRAP, JACKKNIFE Y VALIDACIÓN CRUZADA

- G. Ciaburro, Hand-os Simulations...: Capítulo 6
- Bruce, Bruce & Gedeck, Practical Statistics for Data Scientists: Capítulo 2
- S. M. Ross, Simulation: Sección 8.3

Remuestreo (resampling): Introducción

- Tomamos una muestra a partir de una población
- Calculamos una “estadística” (ej. media)
- ¿Qué tan cerca de la “real” (población) estamos? ¿Intervalo de confianza?
- Sabemos por **Teorema Central del Límite** que si tomo muchas otras muestras y calculo la “estadística” puedo verificar que la distribución de ellas es gaussiana/normal. La media de la muestra sigue una distribución normal.
- También sabemos por la **Ley de Grandes Números** que mientras más grande sea la muestra, esa media de la muestra se va a ir acercando más a la media de la población.
- Importante: que la muestra sea aleatoria, para que sea representativa de la población.

Preguntas sobre la distribución normal de la “estadística”:

- ¿Cuál es la desviación estándar? (error estándar)
- ¿Intervalo de confianza donde encontraremos el 95% de los resultados?

Las técnicas de resampling nos ayudan a calcular esta distribución

Remuestreo (resampling): Introducción

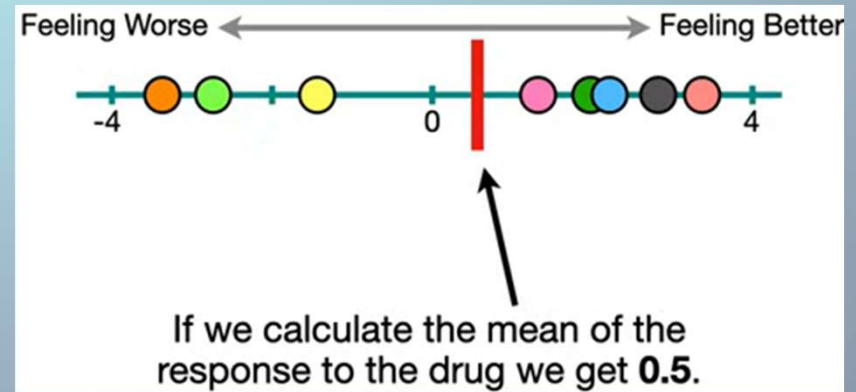
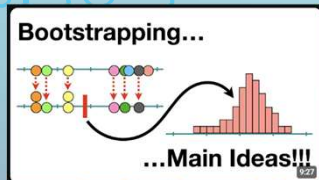
- **Remuestreo**: es el proceso de repetir la toma de muestras de la población.
- **La idea**: realizar el remuestreo pero con la única muestra que tenemos.
- **Lógica**: construir un conjunto de muestras que sean estadísticamente similares a tomar muestras aleatorias de la población.
- El principal requisito es que la muestra original sea “lo suficientemente representativa” de la población
- La idea es tratar de extrapolar lo obtenido con la muestra hacia la población entera. Sirve cuando no puedo, o es muy “caro” obtener muchas muestras de la población.
- Nos permite evaluar los intervalos de confianza y error estándar de la “estadística” calculada

Bootstrap

- Remuestreo: armamos nuevas muestras desde la original, eligiendo elementos al azar “con reemplazo”.
- Con reemplazo: un elemento es elegido, pero luego sigue disponible para ser elegido de nuevo. Así, puede aparecer mas de una vez en la muestra.

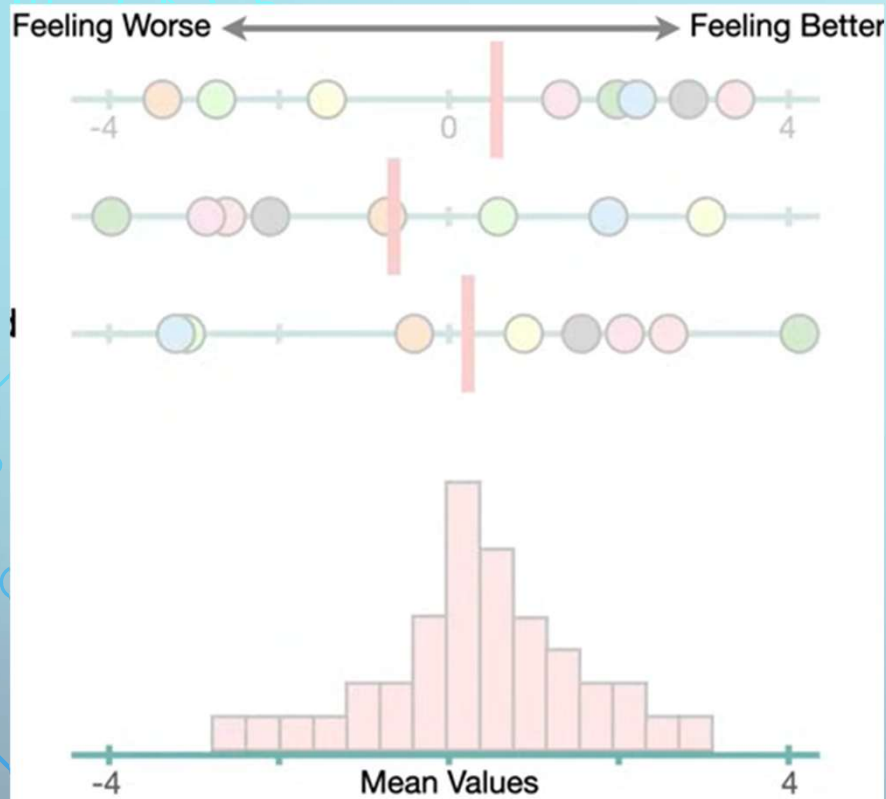
Ejemplo de StatQuest

<https://www.youtube.com/watch?v=Xz0x-8-cgaQ>

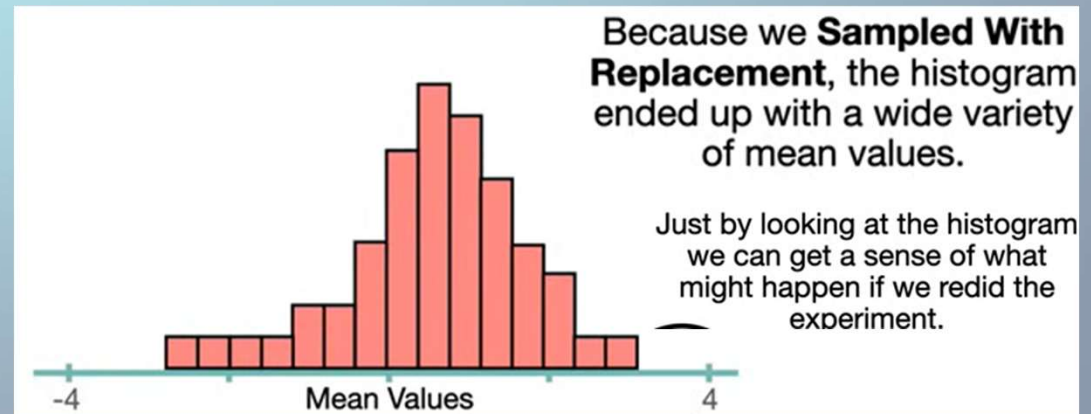
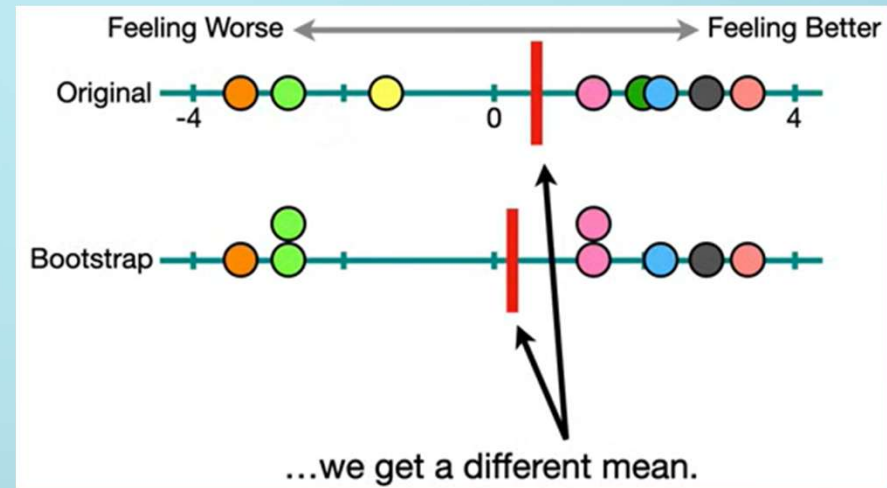


Bootstrap

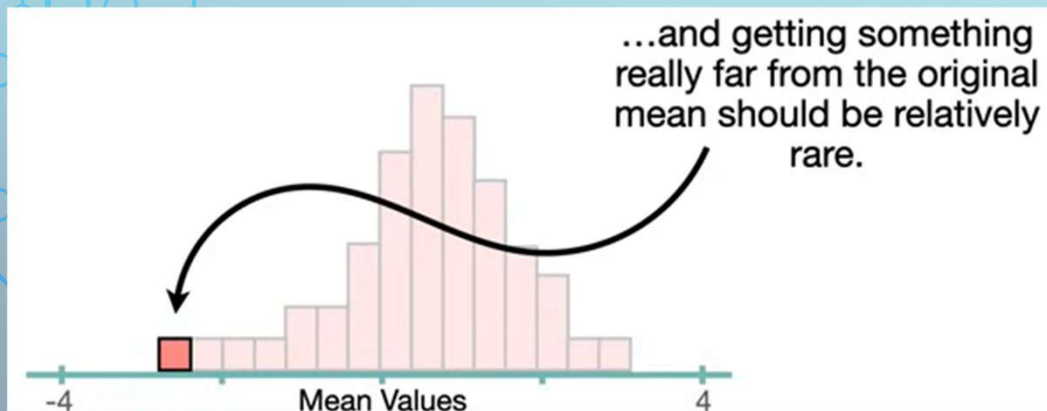
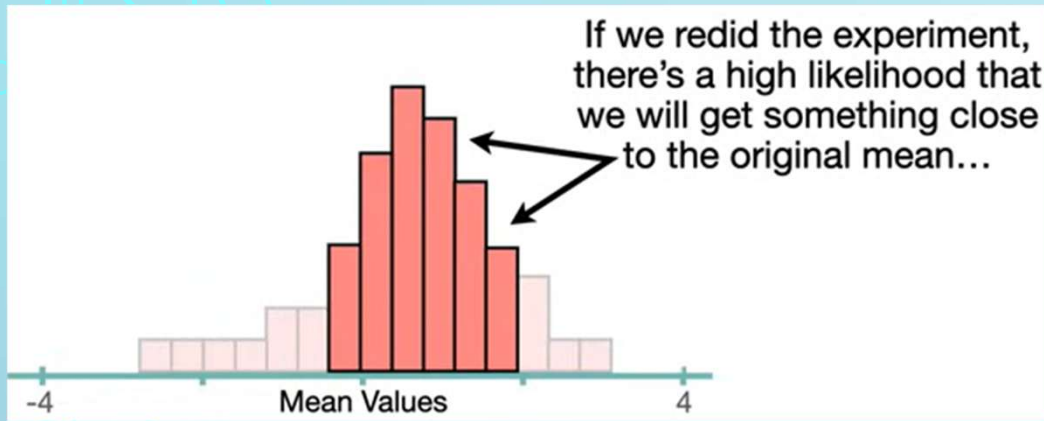
Si pudieramos repetir el experimento



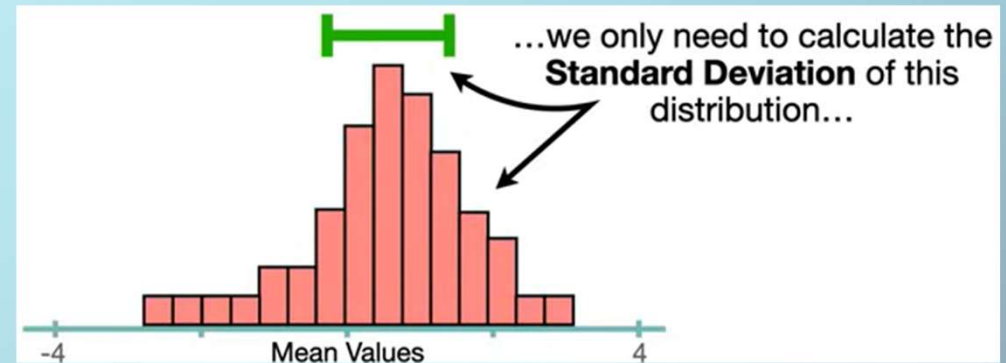
¡Pero no podemos! → Bootstrap



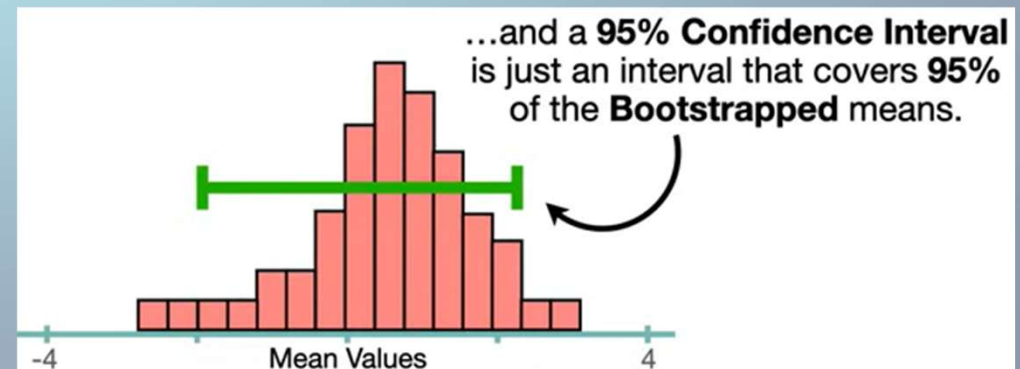
Bootstrap



¿Cuál será el error estándar del valor original de la “estadística”?

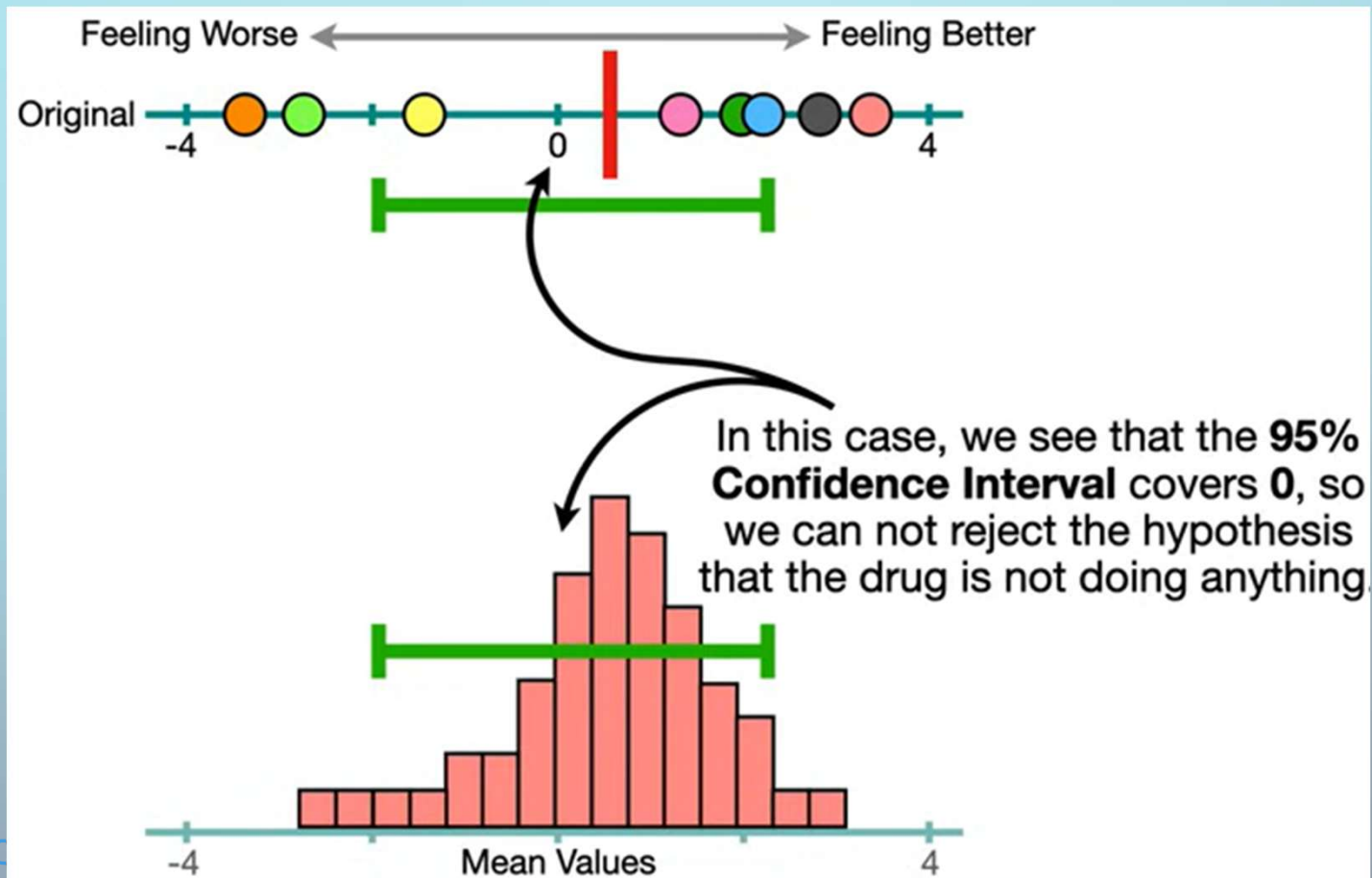


¿y el intervalo de confianza?



Bootstrap

¿Para qué nos sirve?



Bootstrap

Pasos: tengo la muestra original (tiene n elementos)

1. Armamos una muestra bootstrap a partir de la muestra original (n)
2. Calcular la “estadística” (θ) deseada y guardar su valor
3. Repetir B veces
4. Usar la distribución de estos valores para decir/concluir algo

¿Cuál es un buen valor de B ?

- Para error estándar: $\sim 200-500$ suele ser suficiente
- Para intervalos de confianza: $\sim 1000-2000$ es lo recomendado

El costo computacional es de orden $O(B n \text{ costo}(\theta))$

Bootstrap

A tener en cuenta:

- La muestra tiene que ser representativa de la población
(las observaciones tienen que ser independientes e idénticamente distribuidas)
- No funciona bien con muestras pequeñas ($n < 10 \sim 15$)
- Si tenemos series de tiempo: bootstrap en bloque

Ver notebooks sobre bootstrap

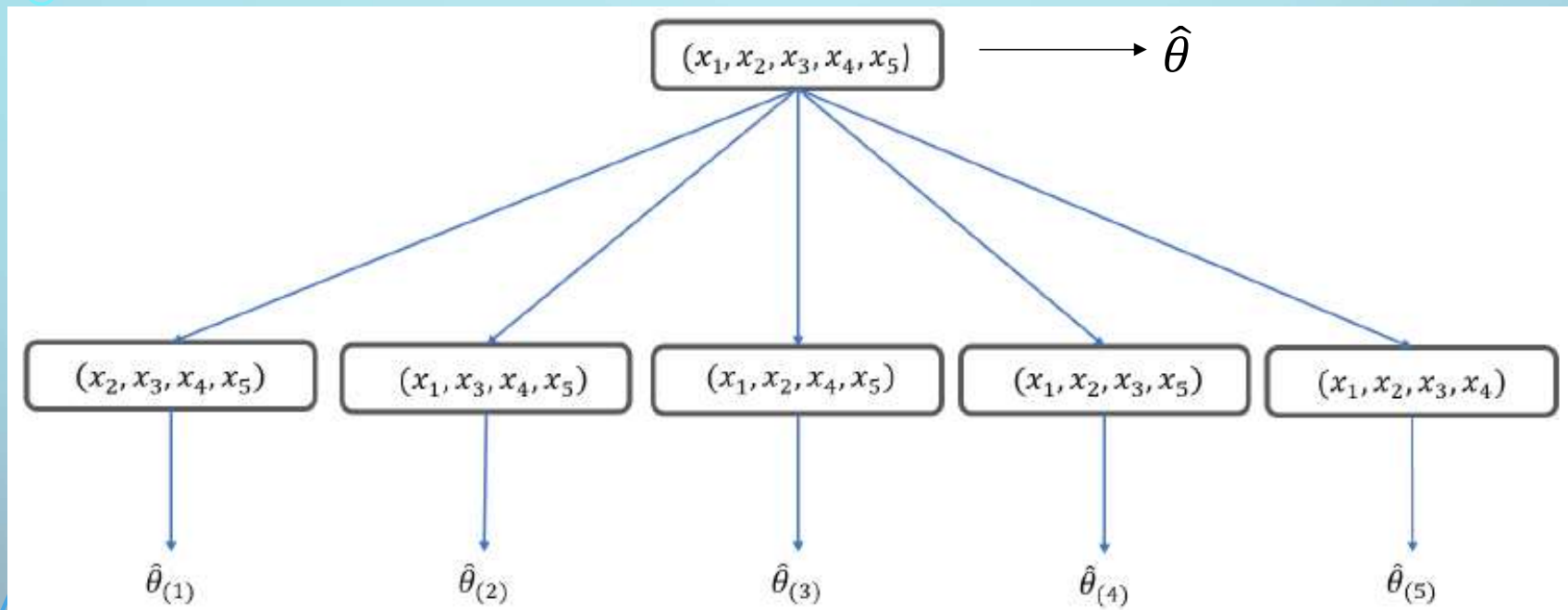
Jackknife

- Idea general: Se trata de quitar observaciones de la muestra y usar lo que queda como nueva muestra para re-evaluar el parámetro estadístico de interés
- Las muestras Jackknife se arman dejando afuera un solo elemento de la muestra. Así, con una muestra de n elementos podré armar otras n muestras Jackknife.

$$x_i = (x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n)$$

Jackknife

Por ejemplo, si tengo una muestra de $n = 5$ elementos $(x_1, x_2, x_3, x_4, x_5)$ con la cual calculo el parámetro estadístico $\hat{\theta}$, me armo entonces:



El estimador Jackknife

$$\hat{\theta}_{(.)} = \frac{1}{n} \sum_{i=1}^n \hat{\theta}_{(i)}$$

(También lo llamamos $\hat{\theta}_{\text{jack}}$)

Jackknife

Qué podemos estimar:

- Varianza:

$$\text{var}_{\text{jack}} = \frac{(n-1)}{n} \sum_i (\hat{\theta}_{(i)} - \hat{\theta}_{\text{jack}})^2$$

- Sesgo (que tan sesgado es nuestro calculo original $\hat{\theta}$):

$$\text{bias}_{\text{jack}} = (n-1) (\hat{\theta}_{\text{jack}} - \hat{\theta})$$

Sesgo “real” sería
 $\hat{\theta} - \theta$

- Si este sesgo es grande (respecto a $\hat{\theta}$), nuestro estimador tiene algún problema sistemático. En este caso podemos construir un *estimador corregido por sesgo*:

$$\hat{\theta}_{\text{jack}}^* = \hat{\theta} - \text{bias}_{\text{jack}} = n \hat{\theta} - (n-1) \hat{\theta}_{\text{jack}}$$

Este estimador tiene sesgo reducido.
Pero ojo, aumenta su varianza

Jackknife

El método Jackknife reduce el bias y evalúa la varianza de un estimador.

A tener en cuenta:

- Funciona bien para estimadores suaves (media, varianza, correlación). Esto es porque el sesgo será pequeño y no necesitaremos utilizar el “estimador corregido por sesgo” (que nos agrega varianza).
- No funciona tan bien para estimadores no suaves (mediana, percentiles extremos). Tendríamos sesgos grandes y valdría la pena corregir el estimador, pero estaríamos aumentando la varianza de la estimación.
- El costo computacional es del orden $O(n)$

ver notebooks

Validación cruzada

- Dado un modelo de machine learning que hemos entrenado ¿cuál es su error real en datos nuevos?
- ¿Cómo evitamos ser “engañados” por la aleatoriedad? ¿Qué pasa si por casualidad tenemos un modelo que parece andar bien pero generaliza mal (overfitting)?

Repasando:

- Tiene sentido separar aparte una porción de los datos, no usarla en el fiteo/entrenamiento, y luego aplicarle el modelo ya entrenado para ver cuán bien aproxima o estima.



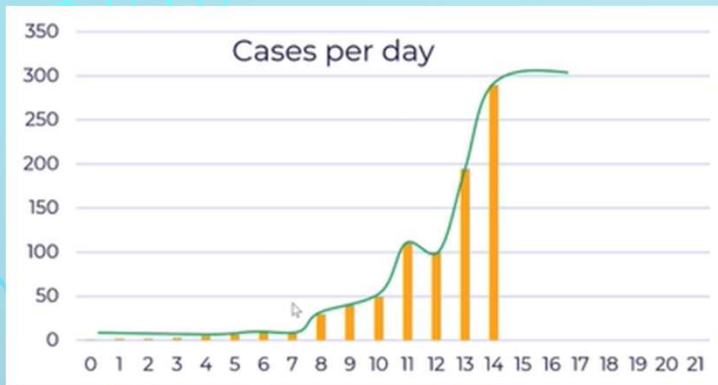
- Si queremos seleccionar entre varios modelos o tunear hiperparámetros, en el entrenamiento separamos aparte otra porción de datos que usaremos para validar los modelos



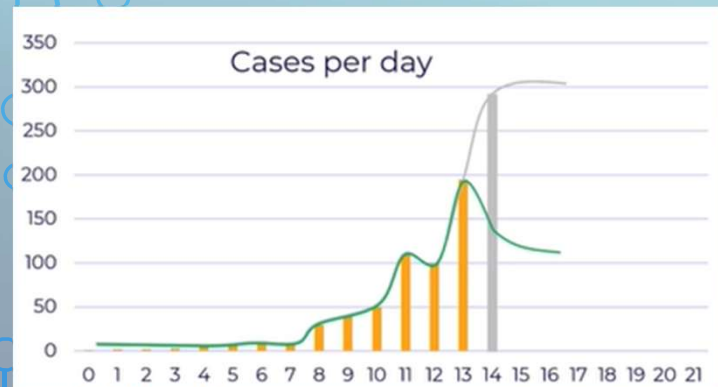
¿Cuán diferente será el resultado si separamos una porción diferente?

Validación cruzada

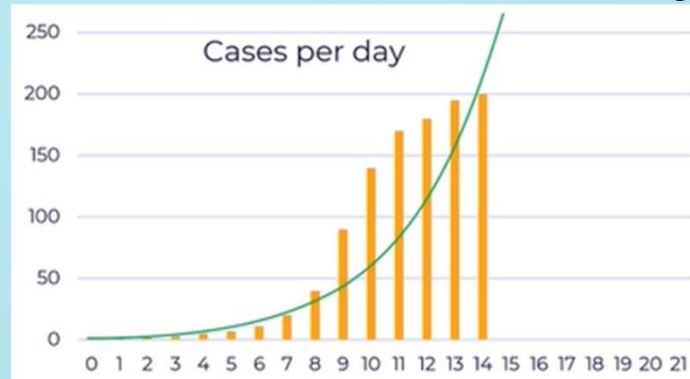
Problema del Overfitting



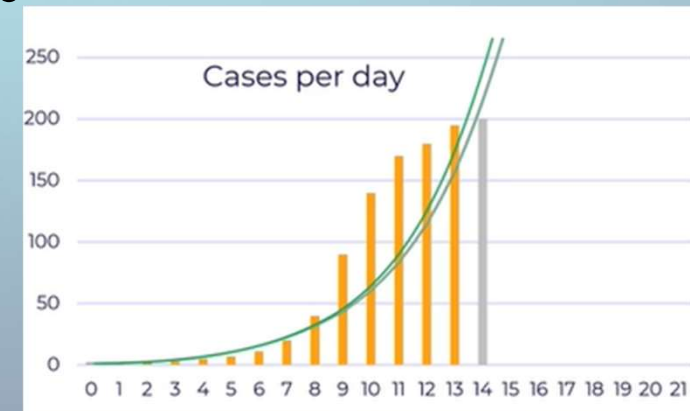
Alta varianza, modelo memoriza datos y generaliza mal



Problema del Underfitting



Alto sesgo, modelo muy simple y generaliza mal



¿Cómo minimizamos la posibilidad de que nos engañe la aleatoriedad?

Validación cruzada



Validación cruzada

K-fold Cross-Validation:

1. Dejar aparte una porción $1/k$ de los datos, para validar
2. Entrenar el modelo con los datos restantes
3. Aplicar el modelo al $1/k$ restante, guardar la métrica/score
4. Volver a unir el $1/k$ y retirar aparte la siguiente porción $1/k$ de datos
5. Repetir 2, 3 y 4 hasta que hayamos usado las k porciones
6. Promediar o combinar de alguna manera los resultados, las métricas de desempeño

Validación cruzada

K-fold Cross-Validation → nos sirve para aproximar, por ej, el error cuadrático medio (MSE) de una técnica de aprendizaje estadístico y así evaluar que tan buen modelo es (que tan buena es su performance)

LOOCV: Leave One Out Cross-Validation → cuando k es el número de datos, es decir, sólo dejamos aparte 1 dato y entrenamos con los $n-1$ datos restantes. Luego repetimos, cambiando el elemento que dejamos afuera.

Validación cruzada

Mucha info en página de scikit-learn

https://scikit-learn.org/stable/modules/cross_validation.html

```
from sklearn.model_selection import cross_val_score
```

```
from sklearn.model_selection import ShuffleSplit
```

```
from sklearn.model_selection import cross_validate
```

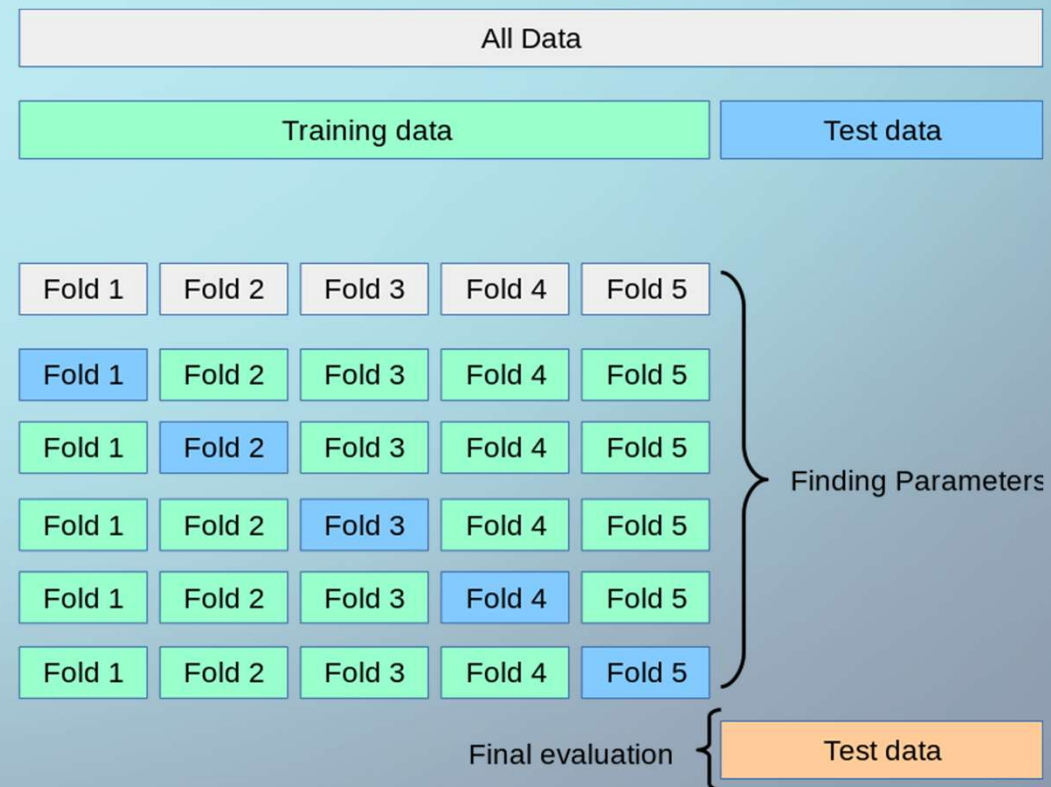
```
from sklearn.model_selection import LeaveOneOut
```

```
from sklearn.model_selection import LeavePOut
```

```
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.model_selection import RandomizedSearchCV
```

OJO: Si hay desbalanceo de clases → Cross-Validation with Stratification



Validación cruzada

Ver notebooks



FIN